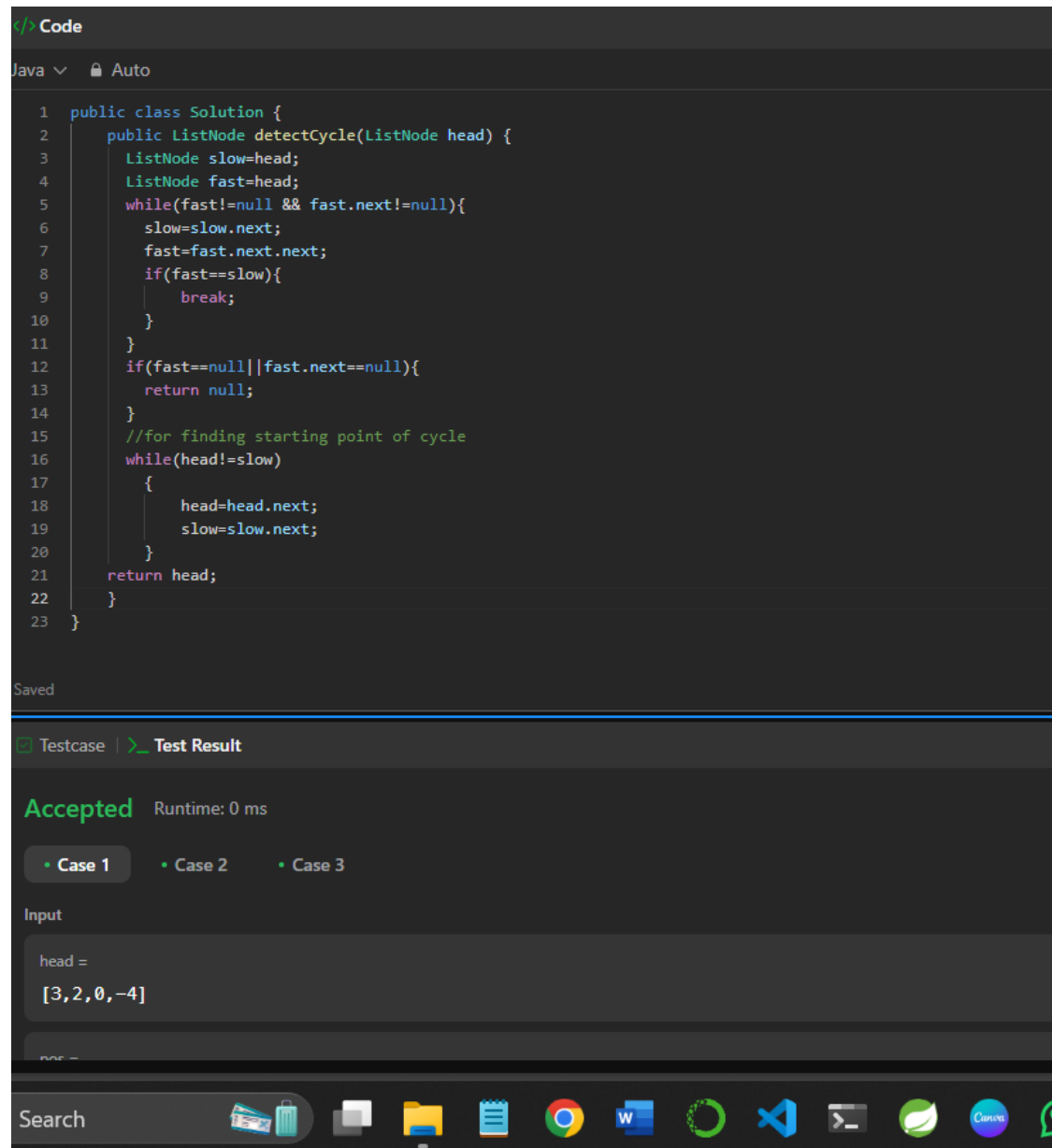


Day 4

Assignment

1] <https://leetcode.com/problems/linked-list-cycle-ii>



```
1 public class Solution {
2     public ListNode detectCycle(ListNode head) {
3         ListNode slow=head;
4         ListNode fast=head;
5         while(fast!=null && fast.next!=null){
6             slow=slow.next;
7             fast=fast.next.next;
8             if(fast==slow){
9                 break;
10            }
11        }
12        if(fast==null||fast.next==null){
13            return null;
14        }
15        //for finding starting point of cycle
16        while(head!=slow)
17        {
18            head=head.next;
19            slow=slow.next;
20        }
21        return head;
22    }
23 }
```

Testcase | **Test Result**

Accepted Runtime: 0 ms

• Case 1 • Case 2 • Case 3

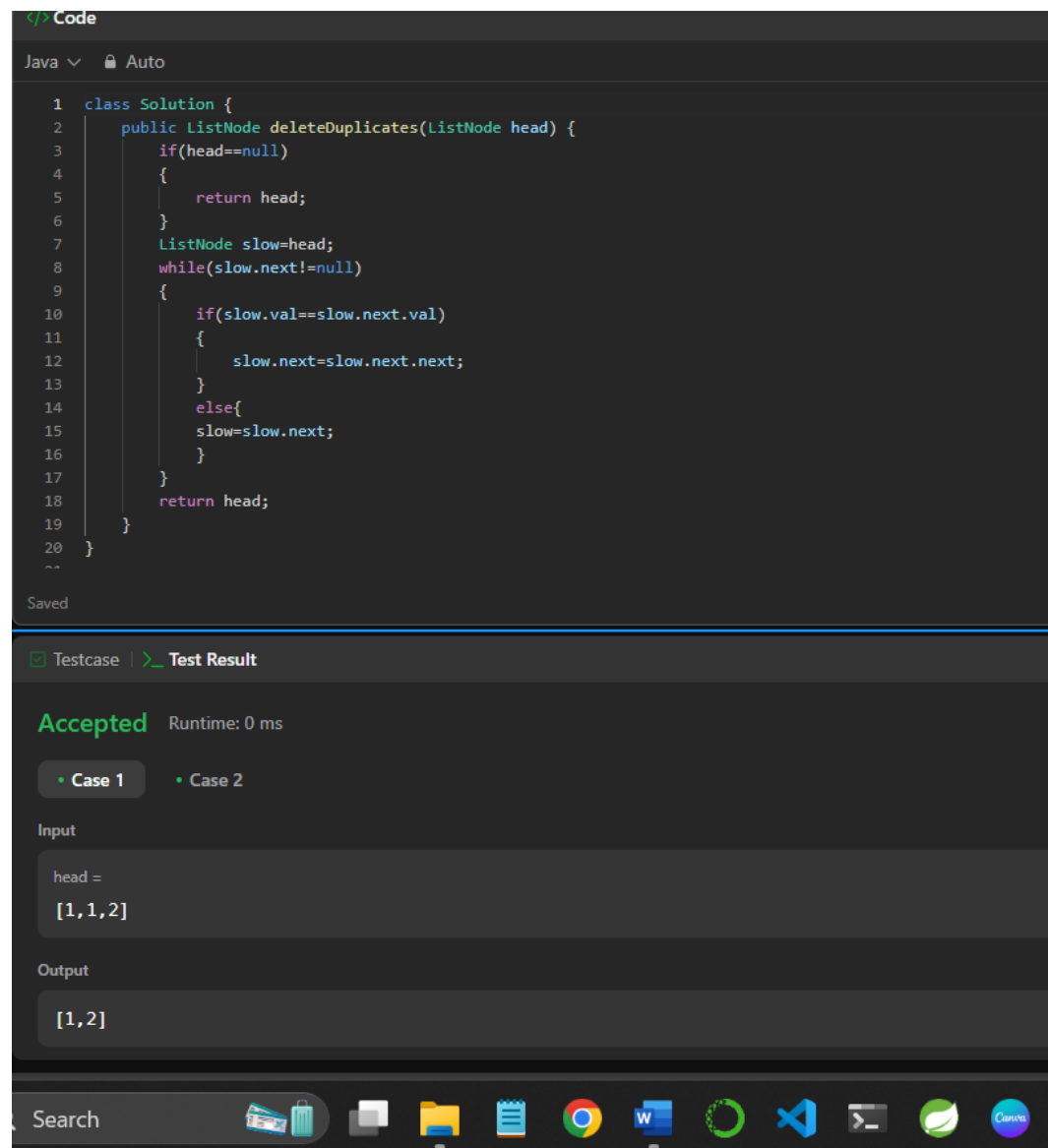
Input

head =

[3,2,0,-4]

Search

2] <https://leetcode.com/problems/remove-duplicates-from-sorted-list>



```
1 class Solution {
2     public ListNode deleteDuplicates(ListNode head) {
3         if(head==null)
4         {
5             return head;
6         }
7         ListNode slow=head;
8         while(slow.next!=null)
9         {
10             if(slow.val==slow.next.val)
11             {
12                 slow.next=slow.next.next;
13             }
14             else{
15                 slow=slow.next;
16             }
17         }
18         return head;
19     }
20 }
```

Accepted Runtime: 0 ms

• Case 1 • Case 2

Input

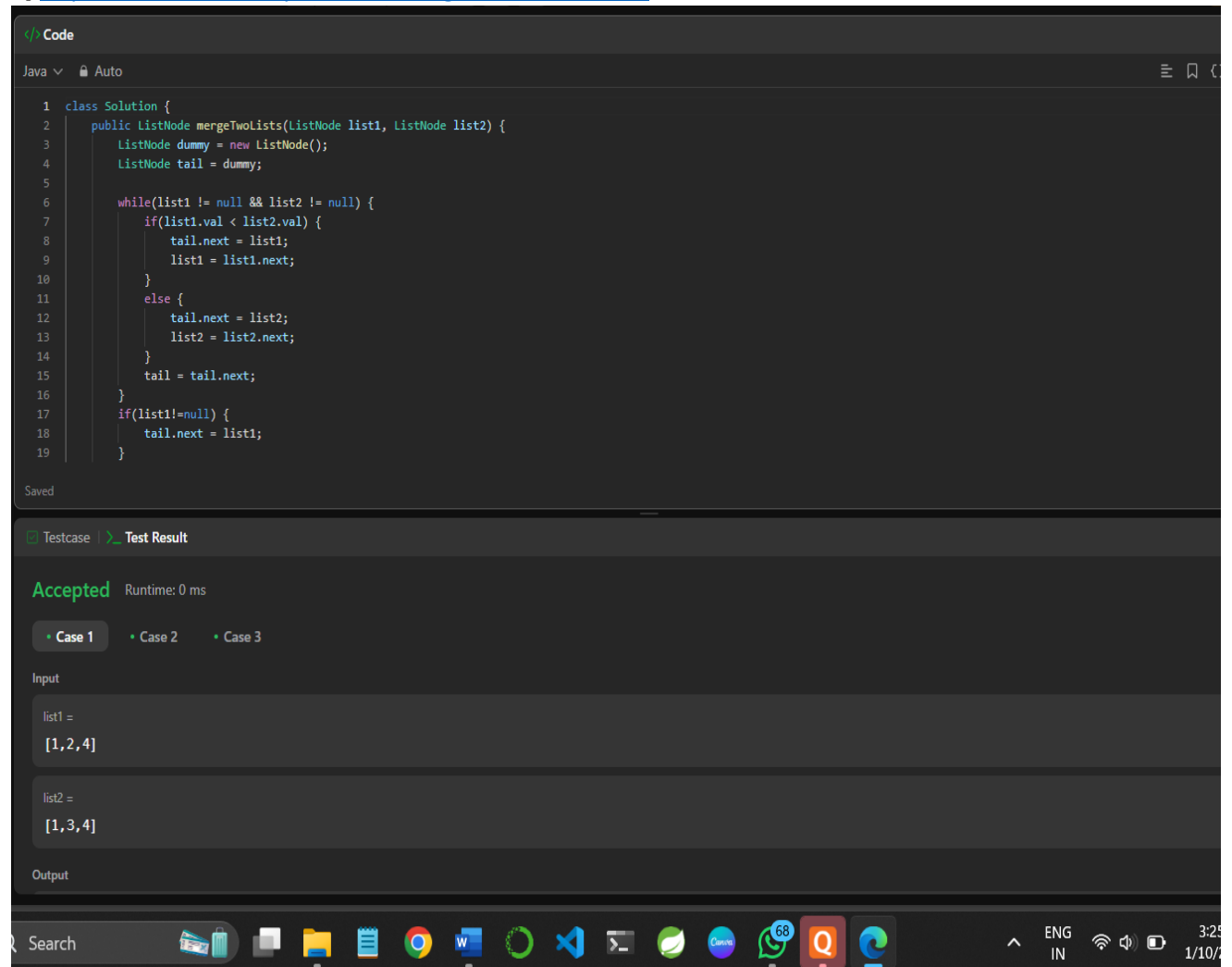
head =
[1,1,2]

Output

[1,2]

Search [taskbar icons]

3] <https://leetcode.com/problems/merge-two-sorted-lists>



The screenshot shows a LeetCode interface with a Java solution for the 'Merge Two Sorted Lists' problem. The code is as follows:

```
1 class Solution {
2     public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
3         ListNode dummy = new ListNode();
4         ListNode tail = dummy;
5
6         while(list1 != null && list2 != null) {
7             if(list1.val < list2.val) {
8                 tail.next = list1;
9                 list1 = list1.next;
10            }
11            else {
12                tail.next = list2;
13                list2 = list2.next;
14            }
15            tail = tail.next;
16        }
17        if(list1 != null) {
18            tail.next = list1;
19        }
20    }
21 }
```

The solution is marked as 'Accepted' with a runtime of 0 ms. Below the code, the 'Testcase' tab is selected, showing 'Case 1' as the active test case. The input is defined as:

list1 = [1,2,4]
list2 = [1,3,4]

The output field is currently empty. The bottom of the image shows a Windows taskbar with various application icons and system status indicators.

4] <https://leetcode.com/problems/swap-nodes-in-pairs>

The screenshot displays a LeetCode solution for the problem "Swap Nodes in Pairs". The code is written in Java and is shown in a dark-themed editor. Below the code, the "Test Result" section shows the input, output, and expected results for a test case.

Code:

```
1 class Solution {
2     public ListNode swapPairs(ListNode head) {
3         if (head == null || head.next == null) return head;
4
5         ListNode dummy = new ListNode(0);
6         dummy.next = head;
7         ListNode prev = dummy;
8
9         while (prev.next != null && prev.next.next != null) {
10             ListNode first = prev.next;
11             ListNode second = prev.next.next;
12
13             first.next = second.next;
14             second.next = first;
15             prev.next = second;
16
17             prev = first;
18         }
19         return dummy.next;
20     }
21 }
```

Test Result:

Testcase | Test Result

Input

head =
[1,2,3,4]

Output

[2,1,4,3]

Expected

[2,1,4,3]

Contribute a testcase

Search

Taskbar icons: Shopping bag, File Explorer, Calendar, Chrome, Word, VS Code, Terminal, WhatsApp, Clideo, Telegram (68 messages).

5] <https://leetcode.com/problems/contains-duplicate/>

```
1 class Solution {
2     public boolean containsDuplicate(int[] nums) {
3         Arrays.sort(nums);
4         int n=nums.length;
5         for(int i=1;i<n;i++)
6         {
7             if(nums[i]==nums[i-1]){
8                 return true;}
9         }
10        return false;
11    }
12 }
```

Saved

☒ Testcase | ☒ Test Result

Accepted Runtime: 0 ms

• **Case 1** • Case 2 • Case 3

Input

nums =
[1,2,3,1]

Output

true

Expected

true

6] <https://leetcode.com/problems/longest-palindrome/>

Java ▾ 🔒 Auto

```
1 class Solution {
2     public int longestPalindrome(String s) {
3         int oddCount = 0; // Count of characters with odd frequency
4         HashMap<Character, Integer> frequencyMap = new HashMap<>();
5
6         for (char ch : s.toCharArray()) {
7             frequencyMap.put(ch, frequencyMap.getOrDefault(ch, 0) + 1);
8             if (frequencyMap.get(ch) % 2 == 0)
9                 oddCount--;
10            else
11                oddCount++;
12        }
13        if (oddCount > 1)
14            return s.length() - oddCount + 1;
15        return s.length();
16    }
17 }
```

Saved

☒ Testcase | >_ Test Result

Accepted Runtime: 0 ms

- Case 1
- Case 2

Input

s =
"abccccdd"

Output

7